Session 26 — Study Guide

What we learned today

1. **A generic, `fields`-array-driven edit panel** — once an input kind and serialization are supported, another field of that kind is usually one fields-array entry.
2. **`@dnd-kit/core`'s `useDraggable` / `useDroppable` pair** — a real draggable card, a real droppable column, matched by a shared id, with no higher-level drag-and-drop framework.
3. **Optimistic UI updates with rollback** — a real local update applied immediately on drop, confirmed silently on success, reverted to a real captured snapshot on failure.
4. **The real Sprint domain's own rules** — at most one real ACTIVE sprint per project, with a friendly service-level `409` and a concurrency-safe PostgreSQL partial unique index; a forward-only `planned -> active -> completed` lifecycle; a completed sprint's fields frozen.
5. **The Backlog as a client-side filter** — issues with no sprint, filtered over data the backend already returns, not through a dedicated backend query parameter.

Why we needed it

Session 25 gave the app a real signed-in user, but Issue Detail was still read-only, and there was no way to visualize or move work through a board or a sprint — the app could show data, but not let a team actually manage it. Today builds the real mechanisms that change that: a generic editing pattern that scales to every future field, a real persisted drag-and-drop board, and a real Sprint/Backlog layer with its own enforced rules.

First-Time Tool Box

Small TypeScript/JavaScript syntax pause: `keyof Issue` means "only property names that exist on `Issue`." `issue[field.key]` reads the property whose name is stored in `field.key`; `{ [field.key]: value }` creates an object with that computed property name. `EditableField` is a discriminated union: after `field.type === "select"`, TypeScript knows that branch has select-only properties such as `options`.

Two Server Action call shapes: `useActionState` with `<form action={...}>` submits `FormData` and receives returned form state. An `onClick`, `onBlur`, or drag handler calls a Server Action directly with ordinary arguments, `await`s it, and handles a thrown error itself.

Tool: `@dnd-kit/core` (this cohort's genuine first real drag-and-drop mechanism).

What it is: a headless drag-and-drop library — it tracks pointer/touch drag state and reports `onDragEnd` with whatever ids the dragged element and the element it was dropped on were given. It renders nothing itself and has no concept of a backend.

Why we use it here: the board's own real use case is the one that justifies a real drag-and-drop mechanism — a card's own column IS its own status, and dragging a card between columns is a real, direct way to change that status, no separate confirmation step needed.

Where/how you run it: `npm install @dnd-kit/core`; then `useDraggable({ id })` on each card, `useDroppable({ id })` on each column, both wrapped in one `<DndContext onDragEnd={...}>`.

Commands/actions needed today:

1. `npm install @dnd-kit/core`.
2. Writing `IssueCard.tsx` (`useDraggable`).
3. Writing `BoardColumn.tsx` (`useDroppable`).
4. Writing `Board.tsx`'s own `handleDragEnd`, with the optimistic update and rollback.

How to tell it worked: dragging a card to a different column moves it immediately, and a reload shows the real backend actually persisted the new status.

Common first-time mistakes: looking up a dropped-on column by array index instead of using the column's own `status` string directly as its droppable id — needlessly reintroduces a lookup `@dnd-kit` was specifically chosen to avoid.

The problem, before the solution

Before today, an issue's `title` / `description` / `status` could only ever be set once, at creation — there was no way to edit anything afterward, no visual board, and no way to group work into sprints at all. Every future editable field would have meant writing a new component from scratch, and "move this to in-progress" meant nothing visual at all.

The solution, step by step

A generic `IssueEditPanel`. WHAT IS IT: one component, driven entirely by a `fields: EditableField[]` prop. WHY: every editable Issue field shares the identical real shape (show a value, let it be edited, save it) — once an input kind and serialization are supported, another field of that kind is usually one array entry. A genuinely new kind still needs type, rendering, and conversion work. WHAT BREAKS WITHOUT IT: every new editable field would mean a new component, duplicating the identical state/save/render logic each time. HOW: `<IssueEditPanel fields={[{ key: "title", label: "Title", type: "text" }, ...]} />`. WHAT TO LOOK AT: `src/components/issues/IssueEditPanel.tsx`. WHAT TO EXPECT: editing the title saves it; adding a new field to the array (homework) needs no new component at all.

`@dnd-kit`'s `useDraggable` / `useDroppable`. WHAT IS IT: two hooks — one marks an element as draggable, the other as a drop target — matched by a shared, meaningful id. WHY: the column's own `status` string doubling as its droppable id means `onDragEnd` can read the new status straight off `event.over.id`, no separate lookup needed. WHAT BREAKS WITHOUT IT: dragging would have no real effect at all; `@dnd-kit` only tracks state and reports ids — it never moves anything itself. HOW: `useDraggable({ id: issue.id })`; `useDroppable({ id: status })`. WHAT TO LOOK AT: `src/components/board/IssueCard.tsx`, `BoardColumn.tsx`. WHAT TO EXPECT: dragging a card visually highlights the column it is over.

Optimistic update with rollback. WHAT IS IT: `setIssues` updates the local list immediately on drop; `updateIssueAction`'s own `.catch()` restores a real captured snapshot if the backend rejects it. WHY: waiting for a network round-trip before a dragged card visually lands would feel broken; real drag-and-drop UIs move the item immediately. WHAT BREAKS WITHOUT IT: a real failed move (e.g. a real `403`) would

leave the UI showing a status the backend never actually accepted. HOW: `const previousIssues = issues; setIssues(...); updateIssueAction(...).catch(() => setIssues(previousIssues));`. WHAT TO LOOK AT: `src/components/board/Board.tsx`. WHAT TO EXPECT: a normal move sticks immediately; simulating a real rejected move snaps the card back to its original column.

The real Sprint domain's own enforced rules. WHAT IS IT: `SprintsService.start()` checks for an existing active sprint and returns a useful domain-specific `409`; a PostgreSQL partial unique index is the final guarantee that a project cannot have two ACTIVE sprints, even when start requests arrive concurrently. WHY: the service check gives the user a clear explanation, while the database remains the final integrity boundary under concurrency. The two layers complement each other. WHAT BREAKS WITHOUT IT: nothing would stop two sprints from being "active" simultaneously, making "what are we working on right now" ambiguous. HOW: `startSprintAction` throws on a real `409`, surfaced inline by `SprintLifecycleControls`. WHAT TO LOOK AT: `src/app/.../sprints/SprintLifecycleControls.tsx`. WHAT TO EXPECT: starting a second sprint while one is active shows a real, specific error message.

Connecting to earlier sessions

The real backend's own `@IsInt()`-validated `PATCH /issues/:id` route already existed (Session

1. — today's own `save()` conversion logic is the identical "convert before sending" reasoning

Session 24's own filters already used for query parameters, now applied to a request body instead. The Backlog's own client-side filter is the identical pattern `SortableIssueList` (Session 24) already used for sorting.

Glossary (required)

- **Generic edit panel:** one component whose rendered fields are entirely driven by a prop passed at the call site, never hard-coded inside the component itself.
- **Droppable / Draggable:** `@dnd-kit`'s own two roles — an element that may receive a drop, and an element that may be dragged — matched by a shared id, never a separate lookup table.
- **Optimistic update:** a local UI change applied immediately, before backend confirmation, reverted only if the backend later rejects it.
- **Forward-only lifecycle:** a real state machine that only ever moves in one direction (`planned -> active -> completed`), enforced server-side, never backward.

Common mistakes

- Writing a new component for a new editable field instead of adding one entry to `IssueEditPanel`'s own `fields` array.
- Looking up a column by array index instead of using its own `status` string as the droppable id directly.
- Capturing `previousIssues` after the optimistic `setIssues` call instead of before — makes the rollback a silent no-op.
- Returning `{ error }` from a directly-called Server Action (`startSprintAction`) instead of throwing — only a thrown error is catchable outside `useActionState`.
- Asking the real backend for a `sprintId` query parameter — no such parameter exists.

Common errors and how to debug them

- **Dragging a card does nothing at all:** confirm both `useDraggable` / `useDroppable` actually received an id, and that `<DndContext onDragEnd={...}>` actually wraps both the cards and the columns — a card dragged outside any `DndContext` reports no events at all.
- **A drag-and-drop move reverts even though it should have succeeded:** confirm the real signed-in user is actually this issue's own reporter/assignee/admin/owner — the real backend's own authorization check rejects everyone else, correctly.
- **Starting a sprint does nothing visible:** confirm `SprintLifecycleControls`'s own `error` state is actually rendered somewhere in the component's JSX.

Never "fix" any of these by removing the backend authorization check, the service-level active sprint check, or the database partial unique index that guarantees the invariant.

Review questions (required)

1. Why does adding a new editable Issue field never require writing a new component?
2. Why does the column's own `status` string double as its droppable id, instead of using a separate, generated id?
3. Why is `previousIssues` captured before the optimistic `setIssues` call, not after?
4. Why do `startSprintAction` / `completeSprintAction` throw instead of returning `{ error }`?

Answers

1. `IssueEditPanel` is driven by its `fields` prop — once an input kind is supported, another field of that kind is usually one array entry; a new kind still needs explicit support.
2. Using the status string directly means `onDragEnd` can read the new status straight off `event.over.id`, with no separate id-to-status lookup table to keep in sync.
3. Capturing it after would already be the new, optimistic value, making a rollback a real no-op — it must be the value from immediately before the change.
4. They are called directly from a button's `onClick`, never through `useActionState` — only a thrown error reaches a direct caller's own `try` / `catch` or `.catch()`.

What you should be able to explain now (required)

- Why a generic, `fields`-driven component scales better than one component per editable field.
- Why `@dnd-kit` needs no separate lookup table when a meaningful id is reused directly.
- Why an optimistic update needs a captured snapshot taken at the right moment to roll back correctly.
- Why the Sprint domain's own rules are enforced server-side, never assumed from the UI alone.

Not yet (required)

- `@mention` autocomplete, Comments, Activity feed, Notifications UI, a cross-workspace Dashboard (Session 27).